

Projeto Semestral: VoltExchange

Sistema de Mercado de Energia P2P (Peer-to-Peer)

Gonalo Marques

Ano Letivo 2025/2026

1 Contexto e Objetivo

O setor energtico est a transitar de um modelo centralizado para um modelo distribuído. O objetivo deste projeto  desenvolver o *backend* completo para a plataforma "**VoltExchange**".

Esta plataforma permite que "Prosumers" (produtores domsticos com painis solares) vendam excedentes de energia a vizinhos. O sistema deve ser robusto, garantindo a integridade financeira das transaes (ACID), a gesto eficiente de milhes de leituras de contadores e a segurana dos dados.

2 Tecnologia e Infraestrutura

- **SGBD Institucional:** O projeto final **tem obrigatoriamente** de ser implementado e alojado no servidor PostgreSQL da Escola.
 - **Acesso:** Cada grupo deve solicitar o acesso e as credenciais de ligao ao docente.
 - O desenvolvimento local  permitido, mas a entrega final e a defesa so realizadas sobre a instncia oficial no servidor da escola.
- **API (Backend):** Linguagem  escolha (Node.js, Python, C#, Java, etc.).
- **Alojamento da API:** A API deve ser alojada num servio Cloud (ex: Vercel, Render, Railway) e configurada para comunicar com o servidor da escola.
- **Segurana:** Uso obrigatrio de *Prepared Statements* (para evitar SQL Injection) e encriptao de passwords (*Hashing*).

Poltica de Autoria e Uso de IA

O uso de ferramentas de IA (ChatGPT, Copilot, etc.)  permitido como auxiliar de produtividade. Contudo, nas defesas e atividades em aula, **qualquer incapacidade de explicar o funcionamento interno**, a lgica de um bloco de cdigo, a escolha de um ndice ou o impacto de uma alterao estrutural resultar em penalizao severa.

Vocs so responsveis por cada linha de cdigo entregue, independentemente de quem (ou o qu) a escreveu.

3 Especificao da Base de Dados

3.1 Estrutura de Dados (Obrigatria)

Os grupos devem respeitar rigorosamente a estrutura definida no **Anexo A**. A existncia das tabelas *OrdensCompra* e *OfertasVenda*  essencial para o funcionamento do motor de matching.

3.2 Lógica de Servidor (PL/pgSQL)

A lógica de negócio deve ser implementada via Stored Procedures no servidor da escola.

Stored Procedure: `sp_MatchingEngine`

Procedimento que processa as ordens pendentes. Deve varrer a tabela `OrdensCompra` e tentar encontrar uma `OfertasVenda` compatível.

- **Critérios:** 1º Preço ($\text{Compra} \geq \text{Venda}$); 2º Proximidade (Mesma região); 3º Data mais antiga.
- Se houver match, gera a transação na tabela `Transacoes` e atualiza os estados das ofertas/ordens.

Desafio

Uma implementação básica executa este procedimento manualmente (em lote). Para a nota máxima nesta componente, o sistema deve ser **Reativo (Event-Driven)**: implementar um **Trigger (AFTER INSERT)** nas tabelas de Ordens e Ofertas que dispare o Matching automaticamente assim que um registo entra, sem intervenção humana.

Transações ACID: `sp_ExecutarCompraDireta`

Usada para compra imediata via API. Recebe um `OfertaID`. Deve garantir atomicidade (ACID): bloquear o registo, verificar se ainda está 'ATIVA', debitar saldo, creditar vendedor e registar transação.

Triggers Obrigatórios

Trigger 1: Analisar inserções na tabela `Leituras`. Se o JSON reportar **anomia** (ver 3.3), alterar o estado do contador para 'MANUTENCAO'.

Trigger 2: Impedir a remoção de utilizadores que tenham saldo positivo ou transações recentes.

3.3 Performance e JSONB

- **Particionamento:** A tabela `Leituras` deve ser particionada (por Data ou Região).
- **Indexação:** Criação de Índices GIN no campo `DadosAudit` (JSONB) para permitir pesquisas performantes.
- **Definição de Anomalia:** Um registo JSON é considerado anómalo se:
 - A chave "`temperatura`" for superior a 80; OU
 - Contiver a chave "`erro_codigo`" (não nula).

4 Desenvolvimento da API

A API (REST) deve expor os serviços da base de dados.

4.1 Requisitos de Segurança

- **Prepared Statements:** Todas as queries SQL na API devem usar parâmetros (ex: `$1`, `$2`) para prevenir *SQL Injection*. Concatenação de strings é proibida.
- **Encriptação:** As passwords devem ser armazenadas com Hashing robusto (ex: `bcrypt`, `Argon2`).

4.2 Endpoints Obrigatórios

- POST /api/auth/register e /login
- POST /api/meters/readings: Recebe payload JSON e grava em DadosAudit.
- GET /api/admin/anomalies: Lista contadores com problemas (deve usar a Query JSONB otimizada).

4.3 Endpoints de Mercado (Trading)

- POST /api/market/buy: Compra imediata (Chama sp_ExecutarCompraDireta).
- POST /api/market/order: Cria uma intenção de compra futura (Insere em OrdensCompra).
- POST /api/market/match: **(Obrigatório para Testes)**
 - Endpoint que dispara manualmente a sp_MatchingEngine.
 - **Nota:** Mesmo que o grupo implemente a solução automática via Triggers (Desafio de Excelência), este endpoint **tem de existir** para permitir ao docente forçar a execução e testar a lógica durante a avaliação.

5 Checkpoints e Avaliação em Sala

O não cumprimento dos checkpoints impede a realização das atividades presenciais.

Checkpoint 1 (8/4 - Estrutura e Dados):

- Acesso ao servidor da escola validado.
- Tabelas do Anexo A criadas.
- **Seeding:** Tabela Leituras com no mínimo **500.000 registos** e OfertasVenda com 1.000 registos.

Checkpoint 2 (13/5 - Lógica e API):

- Stored Procedures e Triggers funcionais.
- API alojada (Vercel/Render) a comunicar com a BD.
- Demonstração de segurança (tentativa de SQL Injection falhada).

6 Entregáveis Finais

A submissão deve ser feita na plataforma de e-learning num único ficheiro ZIP contendo:

1. **Código Fonte:** Ficheiros da API (organizados, sem node_modules).
2. **Scripts SQL:** ddl.sql (Criação), seed.sql (Dados), logic.sql (Procedures).
3. **Relatório Técnico (PDF):** Diagrama ER, Justificação de Índices/Partições, Planos de Execução e Links da API em produção.

A Anexo A: Dicionário de Dados Obrigatório

Nota: Tipos de dados indicativos para PostgreSQL. A nomenclatura das tabelas e colunas deve ser respeitada.

1. Tabela: Utilizadores

- UtilizadorID (PK, Serial)
- Nome (Varchar)
- Email (Varchar, Unique)
- PasswordHash (Varchar - **Nunca guardar em texto limpo**)
- Saldo (Numeric)

2. Tabela: Contadores

- ContadorID (PK, Serial)
- UtilizadorID (FK)
- NumeroSerie (Varchar, Unique)
- Estado (Varchar: 'ATIVO', 'MANUTENCAO')

3. Tabela: Leituras

- LeituraID (PK, BigInt/Serial)
- ContadorID (FK)
- DataHora (Timestamp)
- KWh_Leitura (Numeric)
- DadosAudit (**JSONB**) - *Logs técnicos (ex: temperatura, erros).*

4. Tabela: OfertasVenda

- OfertaID (PK, Serial)
- VendedorID (FK)
- QuantidadeKWh (Numeric)
- PrecoUnitario (Numeric)
- Estado (Varchar: 'ATIVA', 'VENDIDA', 'CANCELADA')
- DataCriacao (Timestamp)

5. Tabela: OrdensCompra (Matching)

- OrdemID (PK, Serial)
- CompradorID (FK)
- QuantidadeKWh (Numeric)
- PrecoMaximo (Numeric)
- Estado (Varchar: 'PENDENTE', 'CONCLUIDA')

6. Tabela: Transacoes

- TransacaoID (PK, Serial)
- OfertaID (FK, Nullable)
- CompradorID (FK)
- VendedorID (FK)
- ValorTotal (Numeric)
- DataTransacao (Timestamp)